

**FUNDACIÓN ESCUELA TECNOLÓGICA DE NEIVA**  
**“JESÚS OVIEDO PÉREZ”**

**TALLER DE LABORATORIO N.º 3**

Procesamiento asíncrono de eventos: API, base de datos y cola de mensajes

*Presentado por:*

**David Felipe Garcia Ortiz**  
**Juan David Claros**

*Presentado a:*

**Ing. Juan Carlos Polania Cortes**

Optativa IV – Sistemas de Tiempo Real Distribuidos  
Ingeniería de Software – Semestre IX  
Periodo 2026-2  
Neiva, septiembre de 2026

## Contenido

1. Descripción de la solución
2. Script de creación de las tablas
3. Entorno Docker
4. Documentación y verificación de la API con Swagger
5. Medición (20 pagos seguidos, acción de 5 s)
6. Casos comprobados (consumidor detenido, acción más lenta, fallo al procesar)
7. Preguntas de cierre
8. Entregables

## 1. Descripción de la solución

Se construyó un flujo de registro de pagos en el que la operación de negocio queda guardada de inmediato y su procesamiento ocurre después, de forma independiente, disparado por un mensaje en una cola. Los cuatro servicios (MySQL, RabbitMQ, la API y el consumidor) se ejecutan en contenedores Docker y se levantan con un único comando.

| Componente | Tecnología                                  | Responsabilidad                                                   |
|------------|---------------------------------------------|-------------------------------------------------------------------|
| MySQL      | mysql:8.4                                   | Tablas pagos y procesamientos (script db/init.sql)                |
| RabbitMQ   | rabbitmq:4-management-alpine                | Cola durable pagos.registrados y cola de fallidos pagos.fallidos  |
| API        | Python 3.12 + FastAPI + aiomysql + aio-pika | POST /pagos, GET /pagos/{id}; documentación Swagger en /docs      |
| Consumidor | Python 3.12 + pika + PyMySQL                | Genera el comprobante (5 s), marca PROCESADO y registra la acción |

### 1.1 Flujo implementado

- **1–2.** El cliente envía el pago; la API lo valida. Si no es válido responde {"status": false, "message": "Datos del pago inválidos"} (HTTP 400) y termina sin insertar ni publicar.
- **3.** La API inserta el pago con estado REGISTRADO y hace COMMIT.
- **4.** Sólo después del COMMIT publica en la cola el mensaje persistente {"pago\_id": id}. El canal usa confirmaciones del publicador (publisher confirms): la API sabe que el mensaje quedó realmente encolado.
- **5.** La API responde de inmediato (HTTP 201) con {"status": true, "message": "Pago registrado", "data": {"id": ..., "estado": "REGISTRADO"}}. No espera al consumidor.
- **6–8.** El consumidor (prefetch = 1) toma el mensaje, registra en su log la hora de toma, genera un archivo de comprobante tardando 5 s, y en una sola transacción pasa el pago a PROCESADO e inserta la fila en procesamientos. Luego registra la hora de fin y, al final, confirma el mensaje (ack).

### 1.2 Decisiones de diseño

- **Confirmación al final:** el ack se envía después del COMMIT. Si el proceso muere antes, RabbitMQ devuelve el mensaje a la cola; si ocurre una excepción, se rechaza (nack sin reencolar) y la dead-letter exchange lo guarda en pagos.fallidos. En ningún caso se pierde.
- **Reintento de conexiones:** la API y el consumidor reintentan la conexión a MySQL y RabbitMQ cada 3 s (hasta 40 intentos); además docker compose espera los healthchecks de ambos servicios.
- **Nombres de servicio:** dentro de la red de Docker los servicios se encuentran como mysql y rabbitmq, nunca como localhost.

- **Idempotencia:** si llega de nuevo un mensaje de un pago ya PROCESADO, el consumidor lo confirma sin repetir la acción.
- **Compensación:** si el pago se guardó pero no se pudo publicar, la API lo elimina y responde status false, para no dejar pagos que nunca se procesarían.

## 2. Script de creación de las tablas

Archivo db/init.sql. MySQL lo ejecuta automáticamente la primera vez que se crea el volumen de datos. El campo estado es un ENUM con los valores REGISTRADO y PROCESADO; las fechas usan DATETIME(3) para medir con milisegundos. Además de los campos mínimos, procesamientos guarda fecha\_toma (hora en que el consumidor tomó el mensaje).

```
-- =====
-- Taller de Laboratorio N.º 3 - Procesamiento asíncrono de eventos
-- Script de creación de las tablas pagos y procesamientos
-- Autores: David Felipe Garcia Ortiz - Juan David Claros
-- MySQL 8.x lo ejecuta automáticamente la primera vez que se crea
-- el volumen de datos (/docker-entrypoint-initdb.d).
-- =====

CREATE DATABASE IF NOT EXISTS pagos_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;

USE pagos_db;

-- Pago registrado por la API. Nace en REGISTRADO y el consumidor
-- lo pasa a PROCESADO cuando termina la acción de procesamiento.
CREATE TABLE IF NOT EXISTS pagos (
  id                INT UNSIGNED NOT NULL AUTO_INCREMENT,
  referencia        VARCHAR(50) NOT NULL,
  valor             DECIMAL(14,2) NOT NULL,
  medio_pago        VARCHAR(30) NOT NULL,
  fecha_registro    DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  estado            ENUM('REGISTRADO', 'PROCESADO') NOT NULL DEFAULT 'REGISTRADO',
  PRIMARY KEY (id),
  INDEX idx_pagos_estado (estado),
  CONSTRAINT chk_pagos_valor CHECK (valor > 0)
) ENGINE = InnoDB;

-- Registro de cada acción de procesamiento ejecutada por el consumidor.
-- fecha_toma: hora en que el consumidor tomó el mensaje de la cola.
-- fecha_procesamiento: hora en que terminó el procesamiento.
CREATE TABLE IF NOT EXISTS procesamientos (
  id                INT UNSIGNED NOT NULL AUTO_INCREMENT,
  pago_id           INT UNSIGNED NOT NULL,
  fecha_toma        DATETIME(3) NULL,
  fecha_procesamiento DATETIME(3) NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  resultado         VARCHAR(255) NOT NULL,
  PRIMARY KEY (id),
  INDEX idx_procesamientos_pago (pago_id),
```

```
CONSTRAINT fk_procesamientos_pago FOREIGN KEY (pago_id) REFERENCES pagos (id)
) ENGINE = InnoDB;
```

### 3. Entorno Docker

El archivo `docker-compose.yml` define los cuatro servicios, sus healthchecks, volúmenes y puertos; la API y el consumidor tienen su propio Dockerfile basado en `python:3.12-slim`. En el equipo sólo se requiere Docker. El entorno completo se levanta con:

```
docker compose up -d --build
```

```

90 C[Laboratory]labPagesAnonymous docker compose up -d -build
91 [internal] load build definition from Dockerfile
92 --> Loading from stage 1:lab08 0.0 done
93 [internal] load build definition from Dockerfile
94 --> Transferring dockerfile: 115B 0.1s done
95 DONE 0.1s
96 [consumer internal] load build definition from Dockerfile
97 --> Transferring dockerfile: 256B 0.1s done
98 DONE 0.1s
99 [internal] load metadata for docker.io/library/python:3.12-slim
100 DONE 0.2s
101 [internal] load dockerimage
102 --> Transferring layers 0: 2B 0.0s done
103 DONE 0.1s
104 [consumer internal] load dockerimage
105 --> Transferring layers 0: 2B 0.0s done
106 DONE 0.1s
107 [LVS] RMM docker: /usr/local/bin/python:3.12-slimsha256:77f0b6a36e6f72068083a6831f386b38f20a312
108 --> Transferring sha256: a37f0b6a36e6f72068083a6831f386b38f20a312 0.1s done
109 DONE 0.1s
110 [consumer internal] load build context
111 --> Transferring context: 9.22B 0.0s done
112 DONE 0.1s
113 [internal] load build context
114 --> Transferring context: 0.0B 0.1s done
115 DONE 0.1s
116 [LVS] COPY requirements.txt
117 CACHED
118 [LVS] RUN pip install -r requirements.txt
119 CACHED
120 [LVS] COPY app.py
121 CACHED
122 [LVS] COMMIT app
123 CACHED
124 [consumer LVS] COPY requirements.txt
125 CACHED
126 [consumer LVS] RUN pip install -r requirements.txt
127 CACHED
128 [internal] exporting to image
129 --> Exporting layers 0: 45B 0.0s done
130 --> Transferring sha256: 7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
131 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
132 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
133 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
134 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
135 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
136 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
137 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
138 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
139 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
140 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
141 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
142 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
143 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
144 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
145 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
146 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
147 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
148 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
149 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
150 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
151 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
152 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
153 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
154 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
155 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
156 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
157 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
158 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
159 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
160 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
161 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
162 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
163 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
164 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
165 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
166 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
167 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
168 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
169 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
170 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
171 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
172 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
173 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
174 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
175 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
176 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
177 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
178 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
179 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
180 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
181 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
182 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
183 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
184 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
185 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
186 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
187 --> Transferring manifest list sha256:7820b6a36e6f72068083a6831f386b38f20a312 0.0s done
188 --> Trans
```

**Figura 1.** Levantamiento del entorno desde cero con un único comando

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:55:54
```

```
PS C:\Laboratorio03_Pagos_Asyncronos> docker compose ps -a
```

| NAME           | SERVICE  | STATUS                      | PORTS                                                                                        |
|----------------|----------|-----------------------------|----------------------------------------------------------------------------------------------|
| lab03-api      | api      | Up 44 seconds               | 0.0.0.0:8090->8000/tcp, [::]:8090->8000/tcp                                                  |
| lab03-consumer | consumer | Up 44 seconds               |                                                                                              |
| lab03-mysql    | mysql    | Up About a minute (healthy) | 0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp                                                  |
| lab03-rabbitmq | rabbitmq | Up About a minute (healthy) | 0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp |

**Figura 2.** Los cuatro servicios en ejecución (mysql y rabbitmq en estado healthy)

El instructivo completo para un tercero está en `README.md` (requisitos, comando único, puertos, pruebas, casos, limpieza).

## 4. Documentación y verificación de la API con Swagger

La API se documenta con Swagger (OpenAPI 3.1) generado por FastAPI: Swagger UI en `http://localhost:8090/docs` y la especificación exportada en `swagger/openapi.json`. Swagger describe los endpoints, los esquemas de petición y respuesta, los códigos HTTP y ejemplos de pago válido e inválido, y permite ejecutarlos desde el navegador con Try it out.

| Método | Ruta             | Descripción                                    | Respuestas      |
|--------|------------------|------------------------------------------------|-----------------|
| POST   | /pagos           | Registrar un pago (no espera el procesamiento) | 201 · 400 · 503 |
| GET    | /pagos/{pago_id} | Consultar un pago por su identificador         | 200 · 400 · 404 |
| GET    | /pagos           | Listar los últimos 100 pagos                   | 200             |
| GET    | /health          | Estado de la API                               | 200             |

## API de pagos - Laboratorio 03 1.0.0 OAS 3.1

[/openapi.json](#)

Taller de Laboratorio N.º 3 - **Procesamiento asíncrono de eventos** (Optativa IV, FET).

Autores: David Felipe García Ortiz y Juan David Claros.

El **POST /pagos** guarda el pago en MySQL con estado **REGISTRADO**, publica su id en la cola **pagos.registrados** de RabbitMQ y responde de inmediato, sin esperar al consumidor. El consumidor procesa el mensaje por su cuenta (≈5 s), genera un comprobante y deja el pago en **PROCESADO**.

Para verlo: registre un pago, consúltelo con **GET /pagos/{id}** enseguida (REGISTRADO) y otra vez pasados 10 segundos (PROCESADO).

### Pagos Registro y consulta de pagos

|      |                  |                                        |
|------|------------------|----------------------------------------|
| GET  | /pagos           | Listar los últimos 100 pagos           |
| POST | /pagos           | Registrar un pago                      |
| GET  | /pagos/{pago_id} | Consultar un pago por su identificador |

### Salud Estado de la API

|     |         |                  |
|-----|---------|------------------|
| GET | /health | Estado de la API |
|-----|---------|------------------|

|                     |            |        |
|---------------------|------------|--------|
| Schemas             |            |        |
| Pago >              | Expand all | object |
| PagoRegistrado >    | Expand all | object |
| Procesamiento >     | Expand all | object |
| RespuestaError >    | Expand all | object |
| RespuestaPago >     | Expand all | object |
| RespuestaPagos >    | Expand all | object |
| RespuestaRegistro > | Expand all | object |

Figura 3. Swagger UI: documentación de la API

## 4.1 Registro de un pago válido e inválido

**POST** /pagos Registrar un pago

Valida el pago, lo inserta con estado **REGISTRADO**, publica {"pago\_id": id} en la cola y responde **sin esperar el procesamiento**. Si los datos no son válidos responde **status: false** y no inserta ni publica nada.

Medios válidos: transferencia, tarjeta, efectivo, pse, nequi.

**Parameters** Cancel

No parameters

**Request body** required application/json

**Examples:**

Pago válido

Edit Value | Schema

```
{
  "referencia": "PAG-0001",
  "valor": 125000,
  "medio": "transferencia"
}
```

Execute Clear

**Responses**

**Curl**

```
curl -X 'POST' \
  'http://api:8000/pagos' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "referencia": "PAG-0001",
    "valor": 125000,
    "medio": "transferencia"
  }'
```

**Request URL**

http://api:8000/pagos

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                                           |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 201  | <p><b>Response body</b></p> <pre>{   "status": true,   "message": "Pago registrado",   "data": {     "id": 1,     "estado": "REGISTRADO"   } }</pre> <p><span>Download</span></p> <p><b>Response headers</b></p> <pre>content-length: 81 content-type: application/json date: Mon, 28 Sep 2026 16:56:08 GMT server: uvicorn</pre> |

**Figura 4.** POST /pagos con el ejemplo “Pago válido”: HTTP 201, status true (booleano) y estado REGISTRADO

**POST** /pagos Registrar un pago

Valida el pago, lo inserta con estado **REGISTRADO**, publica {"pago\_id": id} en la cola y responde **sin esperar el procesamiento**. Si los datos no son válidos responde **status: false** y no inserta ni publica nada.

Medios válidos: transferencia, tarjeta, efectivo, pse, nequi.

**Parameters** Cancel

No parameters

**Request body** required application/json

**Examples:**

Pago inválido

Edit Value | Schema

```
{
  "referencia": "",
  "valor": -125000,
  "medio": "bitcoin"
}
```

Execute Clear

**Responses**

**Curl**

```
curl -X 'POST' \
  'http://api:8000/pagos' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "referencia": "",
    "valor": -125000,
    "medio": "bitcoin"
  }'
```

**Request URL**

http://api:8000/pagos

**Server response**

| Code | Details                                                                                                                                                                                                                                                                                                      |
|------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 400  | <p>Error: Bad Request</p> <p><b>Response body</b></p> <pre>{   "status": false,   "message": "Datos del pago inválidos" }</pre> <p><span>Download</span></p> <p><b>Response headers</b></p> <pre>content-length: 54 content-type: application/json date: Mon, 28 Sep 2026 16:56:15 GMT server: uvicorn</pre> |

Figura 5. POST /pagos con el ejemplo "Pago inválido": HTTP 400 y status false; no se inserta ni se publica nada

## 4.2 Consulta del mismo pago antes y después del procesamiento

Se registró el pago 1 y se consultó con GET /pagos/1 inmediatamente: estado **REGISTRADO** y procesamiento null. Pasados diez segundos se consultó de nuevo: estado **PROCESADO**, con fecha de toma 2026-09-28 11:56:09.243 y fecha de procesamiento 2026-09-28 11:56:14.398. Las dos respuestas muestran estados distintos.

GET

/pagos/{pago\_id}

Consultar un pago por su identificador

^

Devuelve el pago con su estado actual. **procesamiento** es **null** mientras el consumidor no lo ha procesado.

Parameters

Cancel

| Name                                                     | Description            |
|----------------------------------------------------------|------------------------|
| <b>pago_id</b> <span>required</span><br>string<br>(path) | Identificador del pago |

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://api:8000/pagos/1' \
  -H 'accept: */*'
```

Request URL

http://api:8000/pagos/1

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <div><div>Response body</div><div><pre>{   "status": true,   "message": "Pago encontrado",   "data": {     "id": 1,     "referencia": "PAG-0001",     "valor": 125000,     "medio": "transferencia",     "fecha registro": "2026-09-28T11:56:09.200",     "estado": "REGISTRADO",     "procesamiento": null   } }</pre></div><div><div>Copy</div><div>Download</div></div></div> <div><div>Response headers</div><div><pre>content-length: 208 content-type: application/json date: Mon, 28 Sep 2026 16:56:10 GMT server: uvicorn</pre></div></div> |

Figura 6. GET /pagos/{id} inmediatamente después de registrar: REGISTRADO

GET /pagos/{pago\_id} Consultar un pago por su identificador

Devuelve el pago con su estado actual. **procesamiento** es **null** mientras el consumidor no lo ha procesado.

**Parameters**

| Name                                          | Description            |
|-----------------------------------------------|------------------------|
| <b>pago_id</b> * required<br>string<br>(path) | Identificador del pago |

1

Execute Clear

**Responses**

Curl

```
curl -X 'GET' \
  'http://api:8000/pagos/1' \
  -H 'accept: */*'
```

Request URL

```
http://api:8000/pagos/1
```

Server response

| Code | Details                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 200  | <p>Response body</p> <pre>{   "status": true,   "message": "Pago encontrado",   "data": {     "id": 1,     "referencia": "PAG-0001",     "valor": 125000,     "medio": "transferencia",     "fecha_registro": "2026-09-28T11:56:09.200",     "estado": "PROCESADO",     "procesamiento": {       "id": 1,       "fecha_toma": "2026-09-28T11:56:09.243",       "fecha_procesamiento": "2026-09-28T11:56:14.398",       "resultado": "Comprobante generado: comprobante_000001.txt"     }   } }</pre> <p>Response headers</p> <pre>content-length: 357 content-type: application/json date: Mon, 28 Sep 2026 16:56:20 GMT server: uvicorn</pre> |

Figura 7. GET /pagos/{id} del mismo pago pasados 10 segundos: PROCESADO

## 5. Medición (20 pagos seguidos, acción de 5 s)

La medición se hizo con herramientas/medir.py, que corre en un contenedor dentro de la red de Docker: envía 20 POST seguidos, mide el tiempo de respuesta de cada uno en el cliente y luego consulta los pagos hasta que todos quedan PROCESADO. Los tiempos de registro y procesamiento se toman de la base de datos (fecha\_registro y fecha\_procesamiento, con milisegundos).

| Medida                                                           | Valor observado |
|------------------------------------------------------------------|-----------------|
| Tiempo de respuesta de la API (media de los 20 registros)        | 21,44 ms        |
| Tiempo de respuesta de la API (máximo observado)                 | 63,39 ms        |
| Tiempo entre el registro y el procesamiento del primer pago      | 5,036 s         |
| Tiempo entre el registro y el procesamiento del último pago      | 101,081 s       |
| Tiempo total hasta que los 20 pagos quedaron en estado PROCESADO | 101,447 s       |

Los 20 POST se enviaron en 0,433 s en total (mínimo 13,67 ms, mediana 18,66 ms por petición). Justo al terminar el envío, los pagos estaban así: 20 en REGISTRADO. El consumidor procesa uno a la vez ( $\approx 5$  s cada uno), por eso el último pago espera en la cola cerca de  $20 \times 5 \text{ s} \approx 100 \text{ s}$  mientras la API ya había respondido a los 20 clientes.

```

Windows PowerShell - Laboratorio03_Pagos_Asinchronos 2026-09-28 11:56:27

PS C:\Laboratorio03_Pagos_Asinchronos> docker compose run --rm herramientas python medir.py --n 20 --prefijo MED5 --etiqueta medicion_5s
Container lab03-mysql Running
Container lab03-rabbitmq Running
Container lab03-api Running
Container lab03-mysql Waiting
Container lab03-rabbitmq Waiting
Container lab03-rabbitmq Healthy
Container lab03-mysql Healthy
Enviando 20 pagos seguidos a http://api:8000/pagos ...
#01 HTTP 201 63.39 ms {"status": true, "message": "Pago registrado", "data": {"id": 2, "estado": "REGISTRADO"}}
#02 HTTP 201 29.18 ms {"status": true, "message": "Pago registrado", "data": {"id": 3, "estado": "REGISTRADO"}}
#03 HTTP 201 14.04 ms {"status": true, "message": "Pago registrado", "data": {"id": 4, "estado": "REGISTRADO"}}
#04 HTTP 201 20.01 ms {"status": true, "message": "Pago registrado", "data": {"id": 5, "estado": "REGISTRADO"}}
#05 HTTP 201 15.31 ms {"status": true, "message": "Pago registrado", "data": {"id": 6, "estado": "REGISTRADO"}}
#06 HTTP 201 19.35 ms {"status": true, "message": "Pago registrado", "data": {"id": 7, "estado": "REGISTRADO"}}
#07 HTTP 201 20.25 ms {"status": true, "message": "Pago registrado", "data": {"id": 8, "estado": "REGISTRADO"}}
#08 HTTP 201 20.80 ms {"status": true, "message": "Pago registrado", "data": {"id": 9, "estado": "REGISTRADO"}}
#09 HTTP 201 25.89 ms {"status": true, "message": "Pago registrado", "data": {"id": 10, "estado": "REGISTRADO"}}
#10 HTTP 201 17.54 ms {"status": true, "message": "Pago registrado", "data": {"id": 11, "estado": "REGISTRADO"}}
#11 HTTP 201 17.98 ms {"status": true, "message": "Pago registrado", "data": {"id": 12, "estado": "REGISTRADO"}}
#12 HTTP 201 15.78 ms {"status": true, "message": "Pago registrado", "data": {"id": 13, "estado": "REGISTRADO"}}
#13 HTTP 201 16.81 ms {"status": true, "message": "Pago registrado", "data": {"id": 14, "estado": "REGISTRADO"}}
#14 HTTP 201 16.05 ms {"status": true, "message": "Pago registrado", "data": {"id": 15, "estado": "REGISTRADO"}}
#15 HTTP 201 17.97 ms {"status": true, "message": "Pago registrado", "data": {"id": 16, "estado": "REGISTRADO"}}
#16 HTTP 201 24.54 ms {"status": true, "message": "Pago registrado", "data": {"id": 17, "estado": "REGISTRADO"}}
#17 HTTP 201 13.67 ms {"status": true, "message": "Pago registrado", "data": {"id": 18, "estado": "REGISTRADO"}}
#18 HTTP 201 20.64 ms {"status": true, "message": "Pago registrado", "data": {"id": 19, "estado": "REGISTRADO"}}
#19 HTTP 201 17.65 ms {"status": true, "message": "Pago registrado", "data": {"id": 20, "estado": "REGISTRADO"}}
#20 HTTP 201 23.05 ms {"status": true, "message": "Pago registrado", "data": {"id": 21, "estado": "REGISTRADO"}}

Tiempo de respuesta de la API: media 21.44 ms | máximo 63.39 ms | mínimo 13.67 ms
Los 20 POST se enviaron en 0.433 s
Estados justo después del envío: {"REGISTRADO": 20}

Esperando a que el consumidor procese todos los pagos ...
11:56:30 procesados 0/20
11:56:32 procesados 0/20
11:56:34 procesados 0/20
11:56:36 procesados 1/20
11:56:39 procesados 1/20
11:56:41 procesados 2/20
11:56:43 procesados 2/20
11:56:45 procesados 2/20
11:56:47 procesados 3/20
11:56:49 procesados 3/20
11:56:51 procesados 4/20
11:56:53 procesados 4/20
11:56:55 procesados 5/20
11:56:57 procesados 5/20
11:56:59 procesados 5/20
11:57:01 procesados 6/20
11:57:04 procesados 6/20
11:57:06 procesados 7/20
11:57:08 procesados 7/20
11:57:10 procesados 7/20
11:57:12 procesados 8/20
11:57:14 procesados 8/20
11:57:16 procesados 9/20
11:57:18 procesados 9/20
11:57:20 procesados 9/20
11:57:22 procesados 10/20
11:57:24 procesados 10/20
11:57:26 procesados 11/20
11:57:28 procesados 11/20
11:57:30 procesados 12/20
11:57:32 procesados 12/20
11:57:34 procesados 12/20
11:57:36 procesados 13/20
11:57:38 procesados 13/20
11:57:40 procesados 13/20
11:57:42 procesados 14/20
11:57:44 procesados 14/20
11:57:46 procesados 15/20
11:57:48 procesados 15/20
11:57:50 procesados 15/20
11:57:53 procesados 16/20
11:57:55 procesados 16/20
11:57:57 procesados 17/20
11:57:59 procesados 17/20
11:58:01 procesados 17/20
11:58:03 procesados 18/20
11:58:05 procesados 18/20
11:58:07 procesados 19/20
11:58:09 procesados 19/20
11:58:11 procesados 19/20
11:58:13 procesados 20/20

id | registro | tomado de la cola | procesado | en cola (s) | registro->procesado (s)
---|---|---|---|---|---
2 | 2026-09-28T11:56:30.076 | 2026-09-28T11:56:30.093 | 2026-09-28T11:56:35.112 | 0.017 | 5.036
3 | 2026-09-28T11:56:30.096 | 2026-09-28T11:56:35.138 | 2026-09-28T11:56:40.172 | 5.042 | 10.076
4 | 2026-09-28T11:56:30.125 | 2026-09-28T11:56:40.195 | 2026-09-28T11:56:45.234 | 10.070 | 15.109
5 | 2026-09-28T11:56:30.142 | 2026-09-28T11:56:45.263 | 2026-09-28T11:56:50.315 | 15.121 | 20.173
6 | 2026-09-28T11:56:30.161 | 2026-09-28T11:56:50.363 | 2026-09-28T11:56:55.423 | 20.202 | 25.262
7 | 2026-09-28T11:56:30.176 | 2026-09-28T11:56:55.438 | 2026-09-28T11:57:00.460 | 25.262 | 30.284
8 | 2026-09-28T11:56:30.196 | 2026-09-28T11:57:00.477 | 2026-09-28T11:57:05.537 | 30.281 | 35.341
9 | 2026-09-28T11:56:30.218 | 2026-09-28T11:57:05.559 | 2026-09-28T11:57:10.601 | 35.341 | 40.383
10 | 2026-09-28T11:56:30.237 | 2026-09-28T11:57:10.668 | 2026-09-28T11:57:15.676 | 40.431 | 45.439
11 | 2026-09-28T11:56:30.263 | 2026-09-28T11:57:15.695 | 2026-09-28T11:57:20.701 | 45.432 | 50.436
12 | 2026-09-28T11:56:30.284 | 2026-09-28T11:57:20.722 | 2026-09-28T11:57:25.728 | 50.438 | 55.444
13 | 2026-09-28T11:56:30.300 | 2026-09-28T11:57:25.748 | 2026-09-28T11:57:30.758 | 55.448 | 60.458
14 | 2026-09-28T11:56:30.315 | 2026-09-28T11:57:30.779 | 2026-09-28T11:57:35.798 | 60.464 | 65.483
15 | 2026-09-28T11:56:30.332 | 2026-09-28T11:57:35.820 | 2026-09-28T11:57:40.852 | 65.488 | 70.520
16 | 2026-09-28T11:56:30.346 | 2026-09-28T11:57:41.190 | 2026-09-28T11:57:46.207 | 70.944 | 75.061
17 | 2026-09-28T11:56:30.365 | 2026-09-28T11:57:46.230 | 2026-09-28T11:57:51.239 | 75.865 | 80.874
18 | 2026-09-28T11:56:30.389 | 2026-09-28T11:57:51.254 | 2026-09-28T11:57:56.266 | 80.865 | 85.877
19 | 2026-09-28T11:56:30.403 | 2026-09-28T11:57:56.296 | 2026-09-28T11:58:01.307 | 85.893 | 90.904
20 | 2026-09-28T11:56:30.427 | 2026-09-28T11:58:01.324 | 2026-09-28T11:58:06.496 | 90.897 | 96.069
21 | 2026-09-28T11:56:30.442 | 2026-09-28T11:58:06.510 | 2026-09-28T11:58:11.523 | 96.068 | 101.081

===== TABLA DE MEDICIÓN =====
Tiempo de respuesta de la API (media de los 20 registros): 21.44 ms
Tiempo de respuesta de la API (máximo observado) : 63.39 ms
Tiempo entre registro y procesamiento del primer pago : 5.036 s
Tiempo entre registro y procesamiento del último pago : 101.081 s
Tiempo total hasta que los 20 pagos quedaron PROCESADO : 101.447 s
=====
Resultados guardados en resultados/medicion_5s.json

```

Figura 8. Salida real de la herramienta de medición (20 pagos, acción de 5 s)

Refreshed 2026-09-28 11:56:37 Refresh every 5 seconds

Virtual host All

Cluster rabbit@lab03-rabbitmq

User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

▼ All queues (2)

Pagination

Page 1 of 1 - Filter:  ☐ Regex ?

Displaying 2 items , page size up to: 100

| Overview     |                   |         |                |         | Messages |         |       | Message rates |               |        |
|--------------|-------------------|---------|----------------|---------|----------|---------|-------|---------------|---------------|--------|
| Virtual host | Name              | Type    | Features       | State   | Ready    | Unacked | Total | Incoming      | deliver / get | ack    |
| /            | pagos.fallidos    | classic | D / Args       | running | 0        | 0       | 0     |               |               |        |
| /            | pagos.registrados | classic | D DLX DLK Args | running | 0        | 0       | 0     | 4.0/s         | 0.20/s        | 0.00/s |

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

**Figura 9.** Cola durante la medición: los mensajes esperan en pagos.registrados mientras el consumidor avanza

Windows PowerShell - Laboratorio03\_Pagos\_Asyncronos 2026-09-28 11:56:40

```
PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'MED5%' ORDER BY p.id;"
```

| id | referencia       | valor     | medio_pago    | fecha_registro          | estado     | proc_id | fecha_toma              | fecha_procesamiento     |
|----|------------------|-----------|---------------|-------------------------|------------|---------|-------------------------|-------------------------|
| 2  | MED5-115630-0001 | 101000.00 | transferencia | 2026-09-28 11:56:30.076 | PROCESADO  | 2       | 2026-09-28 11:56:30.093 | 2026-09-28 11:56:35.112 |
| 3  | MED5-115630-0002 | 102000.00 | transferencia | 2026-09-28 11:56:30.096 | PROCESADO  | 3       | 2026-09-28 11:56:35.138 | 2026-09-28 11:56:40.172 |
| 4  | MED5-115630-0003 | 103000.00 | transferencia | 2026-09-28 11:56:30.125 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 5  | MED5-115630-0004 | 104000.00 | transferencia | 2026-09-28 11:56:30.142 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 6  | MED5-115630-0005 | 105000.00 | transferencia | 2026-09-28 11:56:30.161 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 7  | MED5-115630-0006 | 106000.00 | transferencia | 2026-09-28 11:56:30.176 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 8  | MED5-115630-0007 | 107000.00 | transferencia | 2026-09-28 11:56:30.196 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 9  | MED5-115630-0008 | 108000.00 | transferencia | 2026-09-28 11:56:30.218 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 10 | MED5-115630-0009 | 109000.00 | transferencia | 2026-09-28 11:56:30.237 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 11 | MED5-115630-0010 | 110000.00 | transferencia | 2026-09-28 11:56:30.263 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 12 | MED5-115630-0011 | 111000.00 | transferencia | 2026-09-28 11:56:30.284 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 13 | MED5-115630-0012 | 112000.00 | transferencia | 2026-09-28 11:56:30.300 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 14 | MED5-115630-0013 | 113000.00 | transferencia | 2026-09-28 11:56:30.315 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 15 | MED5-115630-0014 | 114000.00 | transferencia | 2026-09-28 11:56:30.332 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 16 | MED5-115630-0015 | 115000.00 | transferencia | 2026-09-28 11:56:30.346 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 17 | MED5-115630-0016 | 116000.00 | transferencia | 2026-09-28 11:56:30.365 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 18 | MED5-115630-0017 | 117000.00 | transferencia | 2026-09-28 11:56:30.389 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 19 | MED5-115630-0018 | 118000.00 | transferencia | 2026-09-28 11:56:30.403 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 20 | MED5-115630-0019 | 119000.00 | transferencia | 2026-09-28 11:56:30.427 | REGISTRADO | NUL     | NUL                     | NUL                     |
| 21 | MED5-115630-0020 | 120000.00 | transferencia | 2026-09-28 11:56:30.442 | REGISTRADO | NUL     | NUL                     | NUL                     |

**Figura 10.** Tabla de pagos segundos después del envío: casi todos siguen REGISTRADO

Windows PowerShell - Laboratorio03\_Pagos\_Asyncronos 2026-09-28 11:58:13

```
PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'MED5%' ORDER BY p.id;"
```

| id | referencia       | valor     | medio_pago    | fecha_registro          | estado    | proc_id | fecha_toma              | fecha_procesamiento     |
|----|------------------|-----------|---------------|-------------------------|-----------|---------|-------------------------|-------------------------|
| 2  | MED5-115630-0001 | 101000.00 | transferencia | 2026-09-28 11:56:30.076 | PROCESADO | 2       | 2026-09-28 11:56:30.093 | 2026-09-28 11:56:35.112 |
| 3  | MED5-115630-0002 | 102000.00 | transferencia | 2026-09-28 11:56:30.096 | PROCESADO | 3       | 2026-09-28 11:56:35.138 | 2026-09-28 11:56:40.172 |
| 4  | MED5-115630-0003 | 103000.00 | transferencia | 2026-09-28 11:56:30.125 | PROCESADO | 4       | 2026-09-28 11:56:40.195 | 2026-09-28 11:56:45.234 |
| 5  | MED5-115630-0004 | 104000.00 | transferencia | 2026-09-28 11:56:30.142 | PROCESADO | 5       | 2026-09-28 11:56:45.263 | 2026-09-28 11:56:50.315 |
| 6  | MED5-115630-0005 | 105000.00 | transferencia | 2026-09-28 11:56:30.161 | PROCESADO | 6       | 2026-09-28 11:56:50.363 | 2026-09-28 11:56:55.423 |
| 7  | MED5-115630-0006 | 106000.00 | transferencia | 2026-09-28 11:56:30.176 | PROCESADO | 7       | 2026-09-28 11:56:55.438 | 2026-09-28 11:57:00.460 |
| 8  | MED5-115630-0007 | 107000.00 | transferencia | 2026-09-28 11:56:30.196 | PROCESADO | 8       | 2026-09-28 11:57:00.477 | 2026-09-28 11:57:05.537 |
| 9  | MED5-115630-0008 | 108000.00 | transferencia | 2026-09-28 11:56:30.218 | PROCESADO | 9       | 2026-09-28 11:57:05.559 | 2026-09-28 11:57:10.601 |
| 10 | MED5-115630-0009 | 109000.00 | transferencia | 2026-09-28 11:56:30.237 | PROCESADO | 10      | 2026-09-28 11:57:10.668 | 2026-09-28 11:57:15.676 |
| 11 | MED5-115630-0010 | 110000.00 | transferencia | 2026-09-28 11:56:30.263 | PROCESADO | 11      | 2026-09-28 11:57:15.695 | 2026-09-28 11:57:20.701 |
| 12 | MED5-115630-0011 | 111000.00 | transferencia | 2026-09-28 11:56:30.284 | PROCESADO | 12      | 2026-09-28 11:57:20.722 | 2026-09-28 11:57:25.728 |
| 13 | MED5-115630-0012 | 112000.00 | transferencia | 2026-09-28 11:56:30.300 | PROCESADO | 13      | 2026-09-28 11:57:25.748 | 2026-09-28 11:57:30.758 |
| 14 | MED5-115630-0013 | 113000.00 | transferencia | 2026-09-28 11:56:30.315 | PROCESADO | 14      | 2026-09-28 11:57:30.779 | 2026-09-28 11:57:35.798 |
| 15 | MED5-115630-0014 | 114000.00 | transferencia | 2026-09-28 11:56:30.332 | PROCESADO | 15      | 2026-09-28 11:57:35.820 | 2026-09-28 11:57:40.852 |
| 16 | MED5-115630-0015 | 115000.00 | transferencia | 2026-09-28 11:56:30.346 | PROCESADO | 16      | 2026-09-28 11:57:41.190 | 2026-09-28 11:57:46.207 |
| 17 | MED5-115630-0016 | 116000.00 | transferencia | 2026-09-28 11:56:30.365 | PROCESADO | 17      | 2026-09-28 11:57:46.230 | 2026-09-28 11:57:51.239 |
| 18 | MED5-115630-0017 | 117000.00 | transferencia | 2026-09-28 11:56:30.389 | PROCESADO | 18      | 2026-09-28 11:57:51.254 | 2026-09-28 11:57:56.266 |
| 19 | MED5-115630-0018 | 118000.00 | transferencia | 2026-09-28 11:56:30.403 | PROCESADO | 19      | 2026-09-28 11:57:56.296 | 2026-09-28 11:58:01.307 |
| 20 | MED5-115630-0019 | 119000.00 | transferencia | 2026-09-28 11:56:30.427 | PROCESADO | 20      | 2026-09-28 11:58:01.324 | 2026-09-28 11:58:06.496 |
| 21 | MED5-115630-0020 | 120000.00 | transferencia | 2026-09-28 11:56:30.442 | PROCESADO | 21      | 2026-09-28 11:58:06.510 | 2026-09-28 11:58:11.523 |

**Figura 11.** Tabla de pagos al final: los 20 en PROCESADO con su hora de toma y de procesamiento

```

Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:14

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 42
lab03-consumer | 2026-09-28 11:56:09,249 [CONSUMIDOR] Pago 1 | TOMADO de la cola a las 11:56:09,243 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:14,418 [CONSUMIDOR] Pago 1 | TERMINADO a las 11:56:14,417 (tomado 11:56:09,243, duración 5.174 s) -> PROCESADO | Comprobante generado: comprobante_00
0001.txt
lab03-consumer | 2026-09-28 11:56:30,093 [CONSUMIDOR] Pago 2 | TOMADO de la cola a las 11:56:30,093 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:35,137 [CONSUMIDOR] Pago 2 | TERMINADO a las 11:56:35,128 (tomado 11:56:30,093, duración 5.036 s) -> PROCESADO | Comprobante generado: comprobante_00
0002.txt
lab03-consumer | 2026-09-28 11:56:35,138 [CONSUMIDOR] Pago 3 | TOMADO de la cola a las 11:56:35,138 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:40,193 [CONSUMIDOR] Pago 3 | TERMINADO a las 11:56:40,193 (tomado 11:56:35,138, duración 5.055 s) -> PROCESADO | Comprobante generado: comprobante_00
0003.txt
lab03-consumer | 2026-09-28 11:56:40,194 [CONSUMIDOR] Pago 4 | TOMADO de la cola a las 11:56:40,194 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:45,260 [CONSUMIDOR] Pago 4 | TERMINADO a las 11:56:45,260 (tomado 11:56:40,194, duración 5.066 s) -> PROCESADO | Comprobante generado: comprobante_00
0004.txt
lab03-consumer | 2026-09-28 11:56:45,263 [CONSUMIDOR] Pago 5 | TOMADO de la cola a las 11:56:45,263 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:50,354 [CONSUMIDOR] Pago 5 | TERMINADO a las 11:56:50,353 (tomado 11:56:45,263, duración 5.091 s) -> PROCESADO | Comprobante generado: comprobante_00
0005.txt
lab03-consumer | 2026-09-28 11:56:50,363 [CONSUMIDOR] Pago 6 | TOMADO de la cola a las 11:56:50,363 (redelivered=False)
lab03-consumer | 2026-09-28 11:56:55,436 [CONSUMIDOR] Pago 6 | TERMINADO a las 11:56:55,436 (tomado 11:56:50,363, duración 5.073 s) -> PROCESADO | Comprobante generado: comprobante_00
0006.txt
lab03-consumer | 2026-09-28 11:56:55,437 [CONSUMIDOR] Pago 7 | TOMADO de la cola a las 11:56:55,437 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:00,476 [CONSUMIDOR] Pago 7 | TERMINADO a las 11:57:00,475 (tomado 11:56:55,437, duración 5.037 s) -> PROCESADO | Comprobante generado: comprobante_00
0007.txt
lab03-consumer | 2026-09-28 11:57:00,477 [CONSUMIDOR] Pago 8 | TOMADO de la cola a las 11:57:00,477 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:05,556 [CONSUMIDOR] Pago 8 | TERMINADO a las 11:57:05,555 (tomado 11:57:00,477, duración 5.078 s) -> PROCESADO | Comprobante generado: comprobante_00
0008.txt
lab03-consumer | 2026-09-28 11:57:05,558 [CONSUMIDOR] Pago 9 | TOMADO de la cola a las 11:57:05,558 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:10,667 [CONSUMIDOR] Pago 9 | TERMINADO a las 11:57:10,666 (tomado 11:57:05,558, duración 5.108 s) -> PROCESADO | Comprobante generado: comprobante_00
0009.txt
lab03-consumer | 2026-09-28 11:57:10,668 [CONSUMIDOR] Pago 10 | TOMADO de la cola a las 11:57:10,668 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:15,692 [CONSUMIDOR] Pago 10 | TERMINADO a las 11:57:15,692 (tomado 11:57:10,668, duración 5.024 s) -> PROCESADO | Comprobante generado: comprobante_0
00010.txt
lab03-consumer | 2026-09-28 11:57:15,694 [CONSUMIDOR] Pago 11 | TOMADO de la cola a las 11:57:15,694 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:20,721 [CONSUMIDOR] Pago 11 | TERMINADO a las 11:57:20,720 (tomado 11:57:15,694, duración 5.026 s) -> PROCESADO | Comprobante generado: comprobante_0
00011.txt
lab03-consumer | 2026-09-28 11:57:20,722 [CONSUMIDOR] Pago 12 | TOMADO de la cola a las 11:57:20,721 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:25,746 [CONSUMIDOR] Pago 12 | TERMINADO a las 11:57:25,746 (tomado 11:57:20,721, duración 5.024 s) -> PROCESADO | Comprobante generado: comprobante_0
00012.txt
lab03-consumer | 2026-09-28 11:57:25,748 [CONSUMIDOR] Pago 13 | TOMADO de la cola a las 11:57:25,748 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:30,777 [CONSUMIDOR] Pago 13 | TERMINADO a las 11:57:30,777 (tomado 11:57:25,748, duración 5.029 s) -> PROCESADO | Comprobante generado: comprobante_0
00013.txt
lab03-consumer | 2026-09-28 11:57:30,778 [CONSUMIDOR] Pago 14 | TOMADO de la cola a las 11:57:30,778 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:35,819 [CONSUMIDOR] Pago 14 | TERMINADO a las 11:57:35,818 (tomado 11:57:30,778, duración 5.040 s) -> PROCESADO | Comprobante generado: comprobante_0
00014.txt
lab03-consumer | 2026-09-28 11:57:35,820 [CONSUMIDOR] Pago 15 | TOMADO de la cola a las 11:57:35,820 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:41,189 [CONSUMIDOR] Pago 15 | TERMINADO a las 11:57:41,188 (tomado 11:57:35,820, duración 5.369 s) -> PROCESADO | Comprobante generado: comprobante_0
00015.txt
lab03-consumer | 2026-09-28 11:57:41,190 [CONSUMIDOR] Pago 16 | TOMADO de la cola a las 11:57:41,190 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:46,229 [CONSUMIDOR] Pago 16 | TERMINADO a las 11:57:46,228 (tomado 11:57:41,190, duración 5.039 s) -> PROCESADO | Comprobante generado: comprobante_0
00016.txt
lab03-consumer | 2026-09-28 11:57:46,230 [CONSUMIDOR] Pago 17 | TOMADO de la cola a las 11:57:46,230 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:51,253 [CONSUMIDOR] Pago 17 | TERMINADO a las 11:57:51,253 (tomado 11:57:46,230, duración 5.023 s) -> PROCESADO | Comprobante generado: comprobante_0
00017.txt
lab03-consumer | 2026-09-28 11:57:51,254 [CONSUMIDOR] Pago 18 | TOMADO de la cola a las 11:57:51,254 (redelivered=False)
lab03-consumer | 2026-09-28 11:57:56,295 [CONSUMIDOR] Pago 18 | TERMINADO a las 11:57:56,294 (tomado 11:57:51,254, duración 5.040 s) -> PROCESADO | Comprobante generado: comprobante_0
00018.txt
lab03-consumer | 2026-09-28 11:57:56,296 [CONSUMIDOR] Pago 19 | TOMADO de la cola a las 11:57:56,296 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:01,323 [CONSUMIDOR] Pago 19 | TERMINADO a las 11:58:01,322 (tomado 11:57:56,296, duración 5.027 s) -> PROCESADO | Comprobante generado: comprobante_0
00019.txt
lab03-consumer | 2026-09-28 11:58:01,324 [CONSUMIDOR] Pago 20 | TOMADO de la cola a las 11:58:01,324 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:06,509 [CONSUMIDOR] Pago 20 | TERMINADO a las 11:58:06,509 (tomado 11:58:01,324, duración 5.185 s) -> PROCESADO | Comprobante generado: comprobante_0
00020.txt
lab03-consumer | 2026-09-28 11:58:06,510 [CONSUMIDOR] Pago 21 | TOMADO de la cola a las 11:58:06,509 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:11,551 [CONSUMIDOR] Pago 21 | TERMINADO a las 11:58:11,551 (tomado 11:58:06,509, duración 5.042 s) -> PROCESADO | Comprobante generado: comprobante_0
00021.txt

```

Figura 12. Registro del consumidor: id del pago, hora en que lo tomó de la cola y hora en que terminó

## 6. Casos comprobados

### 6.1 Caso 1: consumidor detenido

Con el consumidor apagado (`docker compose stop consumer`) se registraron cinco pagos. La API respondió con normalidad: HTTP 201 en los cinco, con media de 28,54 ms y máximo de 60,94 ms. Los cinco mensajes quedaron en la cola (`Ready = 5`, `consumidores = 0`) y los pagos en REGISTRADO. Al levantar el consumidor (`docker compose start consumer`), tomó los cinco mensajes y los procesó uno tras otro; los cinco quedaron PROCESADO y la cola vacía.

```

Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:15

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose stop consumer
Container lab03-consumer Stopping
Container lab03-consumer Stopped

```

Figura 13. Se detiene el consumidor

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:16

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose ps -a
NAME                SERVICE    STATUS    PORTS
lab03-api            api        Up 3 minutes    0.0.0.0:8090->8000/tcp, [::]:8090->8000/tcp
lab03-consumer       consumer   Exited (0) 1 second ago
lab03-mysql          mysql      Up 3 minutes (healthy)    0.0.0.0:3307->3306/tcp, [::]:3307->3306/tcp
lab03-rabbitmq       rabbitmq   Up 3 minutes (healthy)    0.0.0.0:5672->5672/tcp, [::]:5672->5672/tcp, 0.0.0.0:15672->15672/tcp, [::]:15672->15672/tcp
```

Figura 14. Estado de los servicios: el consumidor está detenido, la API sigue arriba

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:17

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose run --rm herramientas python medir.py --n 5 --prefijo DET --no-esperar --etiqueta caso1_consumidor_detenido
Container lab03-mysql Running
Container lab03-rabbitmq Running
Container lab03-api Running
Container lab03-mysql Waiting
Container lab03-rabbitmq Waiting
Container lab03-mysql Healthy
Container lab03-rabbitmq Healthy
Enviando 5 pagos seguidos a http://api:8000/pagos ...
#01 HTTP 201 60.94 ms {"status": true, "message": "Pago registrado", "data": {"id": 22, "estado": "REGISTRADO"}}
#02 HTTP 201 23.52 ms {"status": true, "message": "Pago registrado", "data": {"id": 23, "estado": "REGISTRADO"}}
#03 HTTP 201 18.79 ms {"status": true, "message": "Pago registrado", "data": {"id": 24, "estado": "REGISTRADO"}}
#04 HTTP 201 21.79 ms {"status": true, "message": "Pago registrado", "data": {"id": 25, "estado": "REGISTRADO"}}
#05 HTTP 201 17.66 ms {"status": true, "message": "Pago registrado", "data": {"id": 26, "estado": "REGISTRADO"}}

Tiempo de respuesta de la API: media 28.54 ms | máximo 60.94 ms | mínimo 17.66 ms
Los 5 POST se enviaron en 0.144 s
Estados justo después del envío: {'REGISTRADO': 5}

===== TABLA DE MEDICIÓN =====
Tiempo de respuesta de la API (media de los 5 registros): 28.54 ms
Tiempo de respuesta de la API (máximo observado) : 60.94 ms
=====
Resultados guardados en resultados/caso1_consumidor_detenido.json
```

Figura 15. Registro de 5 pagos con el consumidor detenido: la API responde 201 con normalidad

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:22

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name messages_ready messages_unacknowledged consumers
pagos.fallidos 0 0 0
pagos.registrados 5 0 0
```

Figura 16. Cola: 5 mensajes listos y 0 consumidores

Figura 17. Consola de RabbitMQ: 5 mensajes esperando en pagos.registrados

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:38

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'DET%' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 22 | DET-115821-0001 | 101000.00 | transferencia | 2026-09-28 11:58:21.486 | REGISTRADO | NULL | NULL | NULL |
| 23 | DET-115821-0002 | 102000.00 | transferencia | 2026-09-28 11:58:21.513 | REGISTRADO | NULL | NULL | NULL |
| 24 | DET-115821-0003 | 103000.00 | transferencia | 2026-09-28 11:58:21.537 | REGISTRADO | NULL | NULL | NULL |
| 25 | DET-115821-0004 | 104000.00 | transferencia | 2026-09-28 11:58:21.554 | REGISTRADO | NULL | NULL | NULL |
| 26 | DET-115821-0005 | 105000.00 | transferencia | 2026-09-28 11:58:21.575 | REGISTRADO | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Figura 18. Tabla de pagos: los 5 en REGISTRADO, sin procesamiento

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:58:39

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose start consumer
Container lab03-rabbitmq Waiting
Container lab03-mysql Waiting
Container lab03-rabbitmq Healthy
Container lab03-mysql Healthy
Container lab03-consumer Starting
Container lab03-consumer Started
```

Figura 19. Se levanta el consumidor

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:59:16

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 14
lab03-consumer | 2026-09-28 11:58:16,231 [CONSUMIDOR] SIGTERM recibido: el consumidor se detiene
lab03-consumer | 2026-09-28 11:58:42,223 [CONSUMIDOR] Conectado a MySQL (intento 1)
lab03-consumer | 2026-09-28 11:58:42,235 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 11:58:42,259 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=5.0 | SIMULAR_FALLO=False
lab03-consumer | 2026-09-28 11:58:42,259 [CONSUMIDOR] Pago 22 | TOMADO de la cola a las 11:58:42.259 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:47,347 [CONSUMIDOR] Pago 22 | TERMINADO a las 11:58:47.346 (tomado 11:58:42.259, duración 5.088 s) -> PROCESADO | Comprobante generado: comprobante_00022.txt
lab03-consumer | 2026-09-28 11:58:47,350 [CONSUMIDOR] Pago 23 | TOMADO de la cola a las 11:58:47.350 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:52,604 [CONSUMIDOR] Pago 23 | TERMINADO a las 11:58:52.604 (tomado 11:58:47.350, duración 5.254 s) -> PROCESADO | Comprobante generado: comprobante_00023.txt
lab03-consumer | 2026-09-28 11:58:52,605 [CONSUMIDOR] Pago 24 | TOMADO de la cola a las 11:58:52.605 (redelivered=False)
lab03-consumer | 2026-09-28 11:58:57,678 [CONSUMIDOR] Pago 24 | TERMINADO a las 11:58:57.677 (tomado 11:58:52.605, duración 5.072 s) -> PROCESADO | Comprobante generado: comprobante_00024.txt
lab03-consumer | 2026-09-28 11:58:57,680 [CONSUMIDOR] Pago 25 | TOMADO de la cola a las 11:58:57.679 (redelivered=False)
lab03-consumer | 2026-09-28 11:59:02,847 [CONSUMIDOR] Pago 25 | TERMINADO a las 11:59:02.847 (tomado 11:58:57.679, duración 5.168 s) -> PROCESADO | Comprobante generado: comprobante_00025.txt
lab03-consumer | 2026-09-28 11:59:02,848 [CONSUMIDOR] Pago 26 | TOMADO de la cola a las 11:59:02.848 (redelivered=False)
lab03-consumer | 2026-09-28 11:59:08,034 [CONSUMIDOR] Pago 26 | TERMINADO a las 11:59:08.034 (tomado 11:59:02.848, duración 5.185 s) -> PROCESADO | Comprobante generado: comprobante_00026.txt
```

Figura 20. Registro del consumidor al arrancar: toma y procesa los 5 pagos pendientes

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:59:17

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'DET%' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 22 | DET-115821-0001 | 101000.00 | transferencia | 2026-09-28 11:58:21.486 | PROCESADO | 22 | 2026-09-28 11:58:42.259 | 2026-09-28 11:58:47.329 |
| 23 | DET-115821-0002 | 102000.00 | transferencia | 2026-09-28 11:58:21.513 | PROCESADO | 23 | 2026-09-28 11:58:47.351 | 2026-09-28 11:58:52.586 |
| 24 | DET-115821-0003 | 103000.00 | transferencia | 2026-09-28 11:58:21.537 | PROCESADO | 24 | 2026-09-28 11:58:52.606 | 2026-09-28 11:58:57.656 |
| 25 | DET-115821-0004 | 104000.00 | transferencia | 2026-09-28 11:58:21.554 | PROCESADO | 25 | 2026-09-28 11:58:57.680 | 2026-09-28 11:59:02.824 |
| 26 | DET-115821-0005 | 105000.00 | transferencia | 2026-09-28 11:58:21.575 | PROCESADO | 26 | 2026-09-28 11:59:02.849 | 2026-09-28 11:59:08.001 |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

Figura 21. Tabla de pagos: los 5 quedaron PROCESADO

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:59:18

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name      messages_ready  messages_unacknowledged  consumers
pagos.fallidos  0                0                        0
pagos.registrados  0                0                        1
```

Figura 22. Cola vacía y consumidor conectado

RabbitMQ 4.2.4 Erlang 27.3.4.9

Refreshed 2026-09-28 11:59:26 Refresh every 5 seconds

Virtual host All

Cluster rabbit@lab03-rabbitmq

User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

▼ All queues (2)

Pagination

Page 1 of 1 Filter:  ☐ Regex ?

Displaying 2 items, page size up to: 100

| Overview     |                   |         |              |         | Messages |         |       | Message rates |               |        |
|--------------|-------------------|---------|--------------|---------|----------|---------|-------|---------------|---------------|--------|
| Virtual host | Name              | Type    | Features     | State   | Ready    | Unacked | Total | incoming      | deliver / get | ack    |
| /            | pagos.fallidos    | classic | D / Args     | running | 0        | 0       | 0     |               |               |        |
| /            | pagos.registrados | classic | DLX DLK Args | running | 0        | 0       | 0     | 0.00/s        | 0.00/s        | 0.00/s |

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Figura 23. Consola de RabbitMQ al final del caso 1: cola vacía

## 6.2 Caso 2: acción más lenta (15 s)

Se reinició el consumidor con `PROCESS_SECONDS=15` y se repitió la medición de 20 pagos.

| Medida                                   | Acción de 5 s | Acción de 15 s |
|------------------------------------------|---------------|----------------|
| Respuesta de la API: media               | 21,44 ms      | 30,32 ms       |
| Respuesta de la API: máximo              | 63,39 ms      | 77,84 ms       |
| Respuesta de la API: mínimo              | 13,67 ms      | 17,92 ms       |
| Registro → procesamiento del primer pago | 5,036 s       | 15,242 s       |
| Registro → procesamiento del último pago | 101,081 s     | 312,222 s      |
| Tiempo total hasta 20 PROCESADO          | 101,447 s     | 312,779 s      |

El tiempo de respuesta de la API se mantuvo en el mismo orden de magnitud (media 21,44 ms frente a 30,32 ms; la diferencia de 8,88 ms es de milisegundos y del mismo orden que la variación entre corridas —con el consumidor apagado se midieron 28,54 ms—, mientras que la acción se alargó 10 000 ms), mientras que el tiempo hasta el procesamiento del último pago se multiplicó por 3,1 (de 101,081 s a 312,222 s). La lentitud del consumidor sólo alarga la espera en la cola, no la respuesta al cliente.

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 11:59:36
PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 3
lab03-consumer | 2026-09-28 11:59:32,721 [CONSUMIDOR] Conectado a MySQL (intento 1)
lab03-consumer | 2026-09-28 11:59:32,735 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 11:59:35,206 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=15.0 | SIMULAR_FALL0=False
```

**Figura 24.** Consumidor reiniciado con `PROCESS_SECONDS=15`

[illegible]

**Figura 25.** Salida real de la medición con acción de 15 s

Refreshed 2026-09-28 12:00:00 Refresh every 5 seconds

Virtual host All

Cluster rabbit@lab03-rabbitmq

User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

### Queues

▼ All queues (2)

Pagination

Page 1 of 1 Filter:  ☐ Regex ?

Displaying 2 items, page size up to: 100

| Overview     |                          |         |                |         | Messages |         |       | Message rates |               |        |
|--------------|--------------------------|---------|----------------|---------|----------|---------|-------|---------------|---------------|--------|
| Virtual host | Name                     | Type    | Features       | State   | Ready    | Unacked | Total | Incoming      | deliver / get | ack    |
| /            | <b>pagos.fallidos</b>    | classic | D / Args       | running | 0        | 0       | 0     |               |               |        |
| /            | <b>pagos.registrados</b> | classic | D DLX DLK Args | running | 19       | 1       | 20    | 0.00/s        | 0.20/s        | 0.20/s |

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Figura 26. Consola de RabbitMQ durante la medición de 15 s

```
Windows PowerShell - Laboratorio03_Pagos_Asinchronos 2026-09-28 12:00:06

PS C:\Laboratorio03_Pagos_Asinchronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name messages_ready messages_unacknowledged consumers
pagos.fallidos 0 0 0
pagos.registrados 18 1 1
```

Figura 27. Cola durante la medición: 18 listos y 1 sin confirmar (el que se está procesando)

Windows PowerShell - Laboratorio03\_Pagos\_Asinchronos 2026-09-28 12:04:54

```
PS C:\Laboratorio03_Pagos_Asinchronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'LENTO%' ORDER BY p.id;"
```

| id | referencia        | valor     | medio_pago    | fecha_registro          | estado    | proc_id | fecha_toma              | fecha_procesamiento     |
|----|-------------------|-----------|---------------|-------------------------|-----------|---------|-------------------------|-------------------------|
| 27 | LENTO-115940-0001 | 101000.00 | transferencia | 2026-09-28 11:59:40.413 | PROCESADO | 27      | 2026-09-28 11:59:40.465 | 2026-09-28 11:59:55.655 |
| 28 | LENTO-115940-0002 | 102000.00 | transferencia | 2026-09-28 11:59:40.476 | PROCESADO | 28      | 2026-09-28 11:59:55.676 | 2026-09-28 12:00:10.879 |
| 29 | LENTO-115940-0003 | 103000.00 | transferencia | 2026-09-28 11:59:40.509 | PROCESADO | 29      | 2026-09-28 12:00:10.915 | 2026-09-28 12:00:26.241 |
| 30 | LENTO-115940-0004 | 104000.00 | transferencia | 2026-09-28 11:59:40.529 | PROCESADO | 30      | 2026-09-28 12:00:28.910 | 2026-09-28 12:00:44.392 |
| 31 | LENTO-115940-0005 | 105000.00 | transferencia | 2026-09-28 11:59:40.552 | PROCESADO | 31      | 2026-09-28 12:00:51.504 | 2026-09-28 12:01:06.553 |
| 32 | LENTO-115940-0006 | 106000.00 | transferencia | 2026-09-28 11:59:40.572 | PROCESADO | 32      | 2026-09-28 12:01:06.572 | 2026-09-28 12:01:21.600 |
| 33 | LENTO-115940-0007 | 107000.00 | transferencia | 2026-09-28 11:59:40.591 | PROCESADO | 33      | 2026-09-28 12:01:21.740 | 2026-09-28 12:01:36.934 |
| 34 | LENTO-115940-0008 | 108000.00 | transferencia | 2026-09-28 11:59:40.613 | PROCESADO | 34      | 2026-09-28 12:01:36.947 | 2026-09-28 12:01:51.953 |
| 35 | LENTO-115940-0009 | 109000.00 | transferencia | 2026-09-28 11:59:40.637 | PROCESADO | 35      | 2026-09-28 12:01:51.976 | 2026-09-28 12:02:06.998 |
| 36 | LENTO-115940-0010 | 110000.00 | transferencia | 2026-09-28 11:59:40.662 | PROCESADO | 36      | 2026-09-28 12:02:07.023 | 2026-09-28 12:02:22.154 |
| 37 | LENTO-115940-0011 | 111000.00 | transferencia | 2026-09-28 11:59:40.681 | PROCESADO | 37      | 2026-09-28 12:02:22.177 | 2026-09-28 12:02:37.416 |
| 38 | LENTO-115940-0012 | 112000.00 | transferencia | 2026-09-28 11:59:40.703 | PROCESADO | 38      | 2026-09-28 12:02:37.457 | 2026-09-28 12:02:52.526 |
| 39 | LENTO-115940-0013 | 113000.00 | transferencia | 2026-09-28 11:59:40.724 | PROCESADO | 39      | 2026-09-28 12:02:52.589 | 2026-09-28 12:03:07.887 |
| 40 | LENTO-115940-0014 | 114000.00 | transferencia | 2026-09-28 11:59:40.747 | PROCESADO | 40      | 2026-09-28 12:03:07.936 | 2026-09-28 12:03:22.955 |
| 41 | LENTO-115940-0015 | 115000.00 | transferencia | 2026-09-28 11:59:40.769 | PROCESADO | 41      | 2026-09-28 12:03:23.001 | 2026-09-28 12:03:38.045 |
| 42 | LENTO-115940-0016 | 116000.00 | transferencia | 2026-09-28 11:59:40.805 | PROCESADO | 42      | 2026-09-28 12:03:38.061 | 2026-09-28 12:03:53.066 |
| 43 | LENTO-115940-0017 | 117000.00 | transferencia | 2026-09-28 11:59:40.851 | PROCESADO | 43      | 2026-09-28 12:03:53.098 | 2026-09-28 12:04:08.106 |
| 44 | LENTO-115940-0018 | 118000.00 | transferencia | 2026-09-28 11:59:40.885 | PROCESADO | 44      | 2026-09-28 12:04:08.121 | 2026-09-28 12:04:23.127 |
| 45 | LENTO-115940-0019 | 119000.00 | transferencia | 2026-09-28 11:59:40.926 | PROCESADO | 45      | 2026-09-28 12:04:23.141 | 2026-09-28 12:04:38.162 |
| 46 | LENTO-115940-0020 | 120000.00 | transferencia | 2026-09-28 11:59:40.978 | PROCESADO | 46      | 2026-09-28 12:04:38.184 | 2026-09-28 12:04:53.192 |

Figura 28. Tabla de pagos al final del caso 2: los 20 PROCESADO, separados 15 s entre sí

```
Windows PowerShell - Laboratorio03_Pagos_Asincronos 2026-09-28 12:04:56

PS C:\Laboratorio03_Pagos_Asincronos> docker compose logs consumer --tail 42
lab03-consumer | 2026-09-28 11:59:32,735 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 11:59:35,266 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=15.0 | SIMULAR_FALL0=False
lab03-consumer | 2026-09-28 11:59:40,464 [CONSUMIDOR] Pago 27 | TOMADO de la cola a las 11:59:40.464 (redelivered=False)
lab03-consumer | 2026-09-28 11:59:55,674 [CONSUMIDOR] Pago 27 | TERMINADO a las 11:59:55.673 (tomado 11:59:40.464, duración 15.209 s) -> PROCESADO | Comprobante generado: comprobante_000027.txt
lab03-consumer | 2026-09-28 11:59:55,675 [CONSUMIDOR] Pago 28 | TOMADO de la cola a las 11:59:55.675 (redelivered=False)
lab03-consumer | 2026-09-28 12:00:10,914 [CONSUMIDOR] Pago 28 | TERMINADO a las 12:00:10.912 (tomado 11:59:55.675, duración 15.237 s) -> PROCESADO | Comprobante generado: comprobante_000028.txt
lab03-consumer | 2026-09-28 12:00:10,915 [CONSUMIDOR] Pago 29 | TOMADO de la cola a las 12:00:10.915 (redelivered=False)
lab03-consumer | 2026-09-28 12:00:28,907 [CONSUMIDOR] Pago 29 | TERMINADO a las 12:00:28.906 (tomado 12:00:10.915, duración 17.991 s) -> PROCESADO | Comprobante generado: comprobante_000029.txt
lab03-consumer | 2026-09-28 12:00:28,910 [CONSUMIDOR] Pago 30 | TOMADO de la cola a las 12:00:28.910 (redelivered=False)
lab03-consumer | 2026-09-28 12:00:51,501 [CONSUMIDOR] Pago 30 | TERMINADO a las 12:00:51.500 (tomado 12:00:28.910, duración 22.591 s) -> PROCESADO | Comprobante generado: comprobante_000030.txt
lab03-consumer | 2026-09-28 12:00:51,504 [CONSUMIDOR] Pago 31 | TOMADO de la cola a las 12:00:51.504 (redelivered=False)
lab03-consumer | 2026-09-28 12:01:06,569 [CONSUMIDOR] Pago 31 | TERMINADO a las 12:01:06.569 (tomado 12:00:51.504, duración 15.065 s) -> PROCESADO | Comprobante generado: comprobante_000031.txt
lab03-consumer | 2026-09-28 12:01:06,571 [CONSUMIDOR] Pago 32 | TOMADO de la cola a las 12:01:06.571 (redelivered=False)
lab03-consumer | 2026-09-28 12:01:21,739 [CONSUMIDOR] Pago 32 | TERMINADO a las 12:01:21.738 (tomado 12:01:06.571, duración 15.167 s) -> PROCESADO | Comprobante generado: comprobante_000032.txt
lab03-consumer | 2026-09-28 12:01:21,740 [CONSUMIDOR] Pago 33 | TOMADO de la cola a las 12:01:21.740 (redelivered=False)
lab03-consumer | 2026-09-28 12:01:36,946 [CONSUMIDOR] Pago 33 | TERMINADO a las 12:01:36.945 (tomado 12:01:21.740, duración 15.205 s) -> PROCESADO | Comprobante generado: comprobante_000033.txt
lab03-consumer | 2026-09-28 12:01:36,946 [CONSUMIDOR] Pago 34 | TOMADO de la cola a las 12:01:36.946 (redelivered=False)
lab03-consumer | 2026-09-28 12:01:51,972 [CONSUMIDOR] Pago 34 | TERMINADO a las 12:01:51.971 (tomado 12:01:36.946, duración 15.025 s) -> PROCESADO | Comprobante generado: comprobante_000034.txt
lab03-consumer | 2026-09-28 12:01:51,975 [CONSUMIDOR] Pago 35 | TOMADO de la cola a las 12:01:51.975 (redelivered=False)
lab03-consumer | 2026-09-28 12:02:07,021 [CONSUMIDOR] Pago 35 | TERMINADO a las 12:02:07.021 (tomado 12:01:51.975, duración 15.045 s) -> PROCESADO | Comprobante generado: comprobante_000035.txt
lab03-consumer | 2026-09-28 12:02:07,022 [CONSUMIDOR] Pago 36 | TOMADO de la cola a las 12:02:07.022 (redelivered=False)
lab03-consumer | 2026-09-28 12:02:22,176 [CONSUMIDOR] Pago 36 | TERMINADO a las 12:02:22.176 (tomado 12:02:07.022, duración 15.153 s) -> PROCESADO | Comprobante generado: comprobante_000036.txt
lab03-consumer | 2026-09-28 12:02:22,177 [CONSUMIDOR] Pago 37 | TOMADO de la cola a las 12:02:22.177 (redelivered=False)
lab03-consumer | 2026-09-28 12:02:37,454 [CONSUMIDOR] Pago 37 | TERMINADO a las 12:02:37.454 (tomado 12:02:22.177, duración 15.277 s) -> PROCESADO | Comprobante generado: comprobante_000037.txt
lab03-consumer | 2026-09-28 12:02:37,456 [CONSUMIDOR] Pago 38 | TOMADO de la cola a las 12:02:37.456 (redelivered=False)
lab03-consumer | 2026-09-28 12:02:52,587 [CONSUMIDOR] Pago 38 | TERMINADO a las 12:02:52.587 (tomado 12:02:37.456, duración 15.131 s) -> PROCESADO | Comprobante generado: comprobante_000038.txt
lab03-consumer | 2026-09-28 12:02:52,588 [CONSUMIDOR] Pago 39 | TOMADO de la cola a las 12:02:52.588 (redelivered=False)
lab03-consumer | 2026-09-28 12:03:07,935 [CONSUMIDOR] Pago 39 | TERMINADO a las 12:03:07.934 (tomado 12:02:52.588, duración 15.346 s) -> PROCESADO | Comprobante generado: comprobante_000039.txt
lab03-consumer | 2026-09-28 12:03:07,936 [CONSUMIDOR] Pago 40 | TOMADO de la cola a las 12:03:07.936 (redelivered=False)
lab03-consumer | 2026-09-28 12:03:22,998 [CONSUMIDOR] Pago 40 | TERMINADO a las 12:03:22.997 (tomado 12:03:07.936, duración 15.061 s) -> PROCESADO | Comprobante generado: comprobante_000040.txt
lab03-consumer | 2026-09-28 12:03:23,001 [CONSUMIDOR] Pago 41 | TOMADO de la cola a las 12:03:23.001 (redelivered=False)
lab03-consumer | 2026-09-28 12:03:38,058 [CONSUMIDOR] Pago 41 | TERMINADO a las 12:03:38.058 (tomado 12:03:23.001, duración 15.057 s) -> PROCESADO | Comprobante generado: comprobante_000041.txt
lab03-consumer | 2026-09-28 12:03:38,061 [CONSUMIDOR] Pago 42 | TOMADO de la cola a las 12:03:38.060 (redelivered=False)
lab03-consumer | 2026-09-28 12:03:53,097 [CONSUMIDOR] Pago 42 | TERMINADO a las 12:03:53.097 (tomado 12:03:38.060, duración 15.036 s) -> PROCESADO | Comprobante generado: comprobante_000042.txt
lab03-consumer | 2026-09-28 12:03:53,098 [CONSUMIDOR] Pago 43 | TOMADO de la cola a las 12:03:53.098 (redelivered=False)
lab03-consumer | 2026-09-28 12:04:08,119 [CONSUMIDOR] Pago 43 | TERMINADO a las 12:04:08.119 (tomado 12:03:53.098, duración 15.021 s) -> PROCESADO | Comprobante generado: comprobante_000043.txt
lab03-consumer | 2026-09-28 12:04:08,120 [CONSUMIDOR] Pago 44 | TOMADO de la cola a las 12:04:08.120 (redelivered=False)
lab03-consumer | 2026-09-28 12:04:23,140 [CONSUMIDOR] Pago 44 | TERMINADO a las 12:04:23.140 (tomado 12:04:08.120, duración 15.028 s) -> PROCESADO | Comprobante generado: comprobante_000044.txt
lab03-consumer | 2026-09-28 12:04:23,141 [CONSUMIDOR] Pago 45 | TOMADO de la cola a las 12:04:23.141 (redelivered=False)
lab03-consumer | 2026-09-28 12:04:38,182 [CONSUMIDOR] Pago 45 | TERMINADO a las 12:04:38.181 (tomado 12:04:23.141, duración 15.041 s) -> PROCESADO | Comprobante generado: comprobante_000045.txt
lab03-consumer | 2026-09-28 12:04:38,183 [CONSUMIDOR] Pago 46 | TOMADO de la cola a las 12:04:38.183 (redelivered=False)
lab03-consumer | 2026-09-28 12:04:53,204 [CONSUMIDOR] Pago 46 | TERMINADO a las 12:04:53.204 (tomado 12:04:38.183, duración 15.020 s) -> PROCESADO | Comprobante generado: comprobante_000046.txt
```

Figura 29. Registro del consumidor con acción de 15 s

### 6.3 Caso 3: fallo al procesar

Se reinició el consumidor con `SIMULAR_FALL0=true`, que lanza una excepción a mitad de la acción cuando la referencia empieza por `FALLA`. Se registró el pago `FALLA-0001`. El consumidor lo tomó, falló a los 2,5 s, hizo rollback y lo rechazó sin confirmarlo: el mensaje pasó a la cola `pagos.fallidos` (no se perdió) y el pago siguió en `REGISTRADO`, sin fila en procesamientos.

```
Windows PowerShell - Laboratorio03_Pagos_Asincronos 2026-09-28 12:05:03

PS C:\Laboratorio03_Pagos_Asincronos> docker compose logs consumer --tail 3
lab03-consumer | 2026-09-28 12:05:00,012 [CONSUMIDOR] Conectado a MySQL (intento 1)
lab03-consumer | 2026-09-28 12:05:00,032 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 12:05:00,047 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=5.0 | SIMULAR_FALL0=True
```

Figura 30. Consumidor con el fallo simulado activado

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:04

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose run --rm herramientas python registrar.py FALLA-0001
Container lab03-rabbitmq Running
Container lab03-mysql Running
Container lab03-api Running
Container lab03-mysql Waiting
Container lab03-rabbitmq Waiting
Container lab03-rabbitmq Healthy
Container lab03-mysql Healthy
12:05:08 POST /pagos {'referencia': 'FALLA-0001', 'valor': 125000, 'medio': 'transferencia'}
HTTP 201 en 55.95 ms -> {'status': True, 'message': 'Pago registrado', 'data': {'id': 47, 'estado': 'REGISTRADO'}}
```

Figura 31. Registro del pago FALLA-0001: la API responde normalmente

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:13

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 4
lab03-consumer | 2026-09-28 12:05:00.032 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 12:05:00.047 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=5.0 | SIMULAR_FALL0=True
lab03-consumer | 2026-09-28 12:05:00.119 [CONSUMIDOR] Pago 47 | TOMADO de la cola a las 12:05:00.119 (redelivered=False)
lab03-consumer | 2026-09-28 12:05:10.623 [CONSUMIDOR] Pago 47 | ERROR a las 12:05:10.623: Fallo simulado procesando la referencia FALLA-0001 -> mensaje NO confirmado, enviado a pagos.fallidos; el pago sigue REGISTRADO
```

Figura 32. Registro del consumidor: ERROR durante el procesamiento, mensaje no confirmado

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:14

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name      messages_ready  messages_unacknowledged consumers
pagos.fallidos 1           0              0
pagos.registrados 0           0              1
```

Figura 33. Cola: el mensaje quedó en pagos.fallidos

RabbitMQ 4.2.4 Erlang 27.3.4.9

Refreshed 2026-09-28 12:05:22 Refresh every 5 seconds

Virtual host All

Cluster rabbit@lab03-rabbitmq User guest Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

Queues

▼ All queues (2)

Pagination

Page 1 of 1 - Filter:  ☐ Regex ?

Displaying 2 items, page size up to: 100

| Overview     |                   |         |                |         | Messages |         |       | Message rates |               |        |
|--------------|-------------------|---------|----------------|---------|----------|---------|-------|---------------|---------------|--------|
| Virtual host | Name              | Type    | Features       | State   | Ready    | Unacked | Total | Incoming      | deliver / get | ack    |
| /            | pagos.fallidos    | classic | D Args         | running | 1        | 0       | 1     |               |               |        |
| /            | pagos.registrados | classic | D DLX DLK Args | running | 0        | 0       | 0     | 0.00/s        | 0.00/s        | 0.00/s |

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Figura 34. Consola de RabbitMQ: 1 mensaje conservado en pagos.fallidos

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:24

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'FALLA%' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 47 | FALLA-0001 | 125000.00 | transferencia | 2026-09-28 12:05:08.195 | REGISTRADO | NULL | NULL | NULL |
```

Figura 35. Tabla de pagos: FALLA-0001 sigue REGISTRADO

## Variante: el consumidor muere a mitad del procesamiento

Para comprobar que el ack va al final, se registró el pago CAIDA-0001 y se mató el contenedor del consumidor (docker kill) mientras lo procesaba. Antes de matarlo la cola mostraba 1 mensaje sin confirmar; después, como el ack nunca se envió, RabbitMQ lo devolvió a la cola como listo, y el pago quedó en REGISTRADO.

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:25

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose run --rm herramientas python registrar.py CAIDA-0001
Container lab03-mysql Running
Container lab03-rabbitmq Running
Container lab03-api Running
Container lab03-rabbitmq Waiting
Container lab03-mysql Waiting
Container lab03-rabbitmq Healthy
Container lab03-mysql Healthy
12:05:28 POST /pagos {'referencia': 'CAIDA-0001', 'valor': 125000, 'medio': 'transferencia'}
HTTP 201 en 49.56 ms -> {'status': True, 'message': 'Pago registrado', 'data': {'id': 48, 'estado': 'REGISTRADO'}}
```

Figura 36. Registro del pago CAIDA-0001

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:28

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name messages_ready messages_unacknowledged consumers
pagos.fallidos 1 0 0
pagos.registrados 0 1 1
```

Figura 37. Durante el procesamiento: 1 mensaje sin confirmar (unacked)

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:30

PS C:\Laboratorio03_Pagos_Asyncronos> docker kill lab03-consumer
lab03-consumer
```

Figura 38. Se mata el consumidor a mitad del procesamiento

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:31

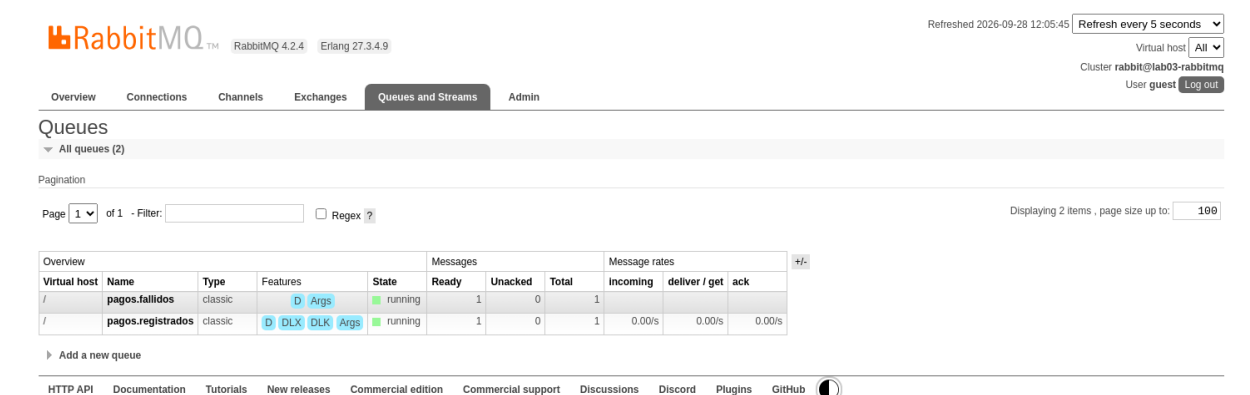
PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 4
lab03-consumer | 2026-09-28 12:05:00,047 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=5.0 | SIMULAR_FALL0=True
lab03-consumer | 2026-09-28 12:05:08,119 [CONSUMIDOR] Pago 47 | TOMADO de la cola a las 12:05:08.119 (redelivered=False)
lab03-consumer | 2026-09-28 12:05:10,623 [CONSUMIDOR] Pago 47 | ERROR a las 12:05:10.623: Fallo simulado procesando la referencia FALLA-0001 -> mensaje NO confirmado, enviado a pagos.
fallidos; el pago sigue REGISTRADO
lab03-consumer | 2026-09-28 12:05:28,056 [CONSUMIDOR] Pago 48 | TOMADO de la cola a las 12:05:28.056 (redelivered=False)
```

Figura 39. Registro del consumidor: tomó el pago 48 pero nunca terminó

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:31

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec rabbitmq rabbitmqctl list_queues name messages_ready messages_unacknowledged consumers
name messages_ready messages_unacknowledged consumers
pagos.fallidos 1 0 0
pagos.registrados 1 0 0
```

Figura 40. Tras la caída: el mensaje volvió a estar listo en pagos.registrados



RabbitMQ 4.2.4 Erlang 27.3.4.9

Refreshed 2026-09-28 12:05:45 Refresh every 5 seconds

Virtual host All

Cluster rabbit@lab03-rabbitmq

User guest Log out

Overview Connections Channels Exchanges Queues and Streams Admin

Queues

▼ All queues (2)

Pagination

Page 1 of 1 - Filter:  ☐ Regex ?

Displaying 2 items, page size up to: 100

| Overview     |                   |         |                                                                                |         |  | Messages |         |       | Message rates |               |        |
|--------------|-------------------|---------|--------------------------------------------------------------------------------|---------|--|----------|---------|-------|---------------|---------------|--------|
| Virtual host | Name              | Type    | Features                                                                       | State   |  | Ready    | Unacked | Total | Incoming      | deliver / get | ack    |
| /            | pagos.fallidos    | classic | <a href="#">D</a> <a href="#">Args</a>                                         | running |  | 1        | 0       | 1     |               |               |        |
| /            | pagos.registrados | classic | <a href="#">D</a> <a href="#">DLX</a> <a href="#">DLK</a> <a href="#">Args</a> | running |  | 1        | 0       | 1     | 0.00/s        | 0.00/s        | 0.00/s |

► Add a new queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

Figura 41. Consola de RabbitMQ tras la caída: el mensaje no se perdió

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:47

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.
id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'CAIDAK' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 48 | CAIDA-0001 | 125000.00 | transferencia | 2026-09-28 12:05:28.040 | REGISTRADO | NULL | NULL | NULL |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

**Figura 42.** *Tabla de pagos: CAIDA-0001 sigue REGISTRADO*

## Recuperación

Se levantó el consumidor sin el fallo simulado: tomó de nuevo el mensaje de CAIDA-0001 (redelivered=True) y lo procesó. Luego se devolvió el mensaje de pagos.fallidos a la cola principal con reprocesar\_fallidos.py y FALLA-0001 también quedó PROCESADO. Ambas colas terminaron vacías.

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:05:59

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec consumer python reprocesar_fallidos.py
Reencolado: {"pago_id": 47}
Mensajes devueltos de pagos.fallidos a pagos.registrados: 1

```

**Figura 43.** *Reencolado del mensaje conservado en pagos.fallidos*

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:06:09

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose logs consumer --tail 8
lab03-consumer | 2026-09-28 12:05:50.379 [CONSUMIDOR] Conectado a MySQL (intento 1)
lab03-consumer | 2026-09-28 12:05:50.389 [CONSUMIDOR] Conectado a RabbitMQ (intento 1)
lab03-consumer | 2026-09-28 12:05:50.407 [CONSUMIDOR] Esperando mensajes en 'pagos.registrados' | PROCESS_SECONDS=5.0 | SIMULAR_FALLO=False
lab03-consumer | 2026-09-28 12:05:50.407 [CONSUMIDOR] Pago 48 | TOMADO de la cola a las 12:05:50.407 (redelivered=True)
lab03-consumer | 2026-09-28 12:05:55.437 [CONSUMIDOR] Pago 48 | TERMINADO a las 12:05:55.436 (tomado 12:05:50.407, duración 5.029 s) -> PROCESADO | Comprobante generado: comprobante_0
00048.txt
lab03-consumer | 2026-09-28 12:06:00.183 [CONSUMIDOR] Pago 47 | TOMADO de la cola a las 12:06:00.182 (redelivered=False)
lab03-consumer | 2026-09-28 12:06:05.229 [CONSUMIDOR] Pago 47 | TERMINADO a las 12:06:05.229 (tomado 12:06:00.182, duración 5.046 s) -> PROCESADO | Comprobante generado: comprobante_0
00047.txt

```

**Figura 44.** *Registro del consumidor: reprocesa el mensaje reentregado y el reencolado*

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:06:10

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.
id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'FALLAK' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 47 | FALLA-0001 | 125000.00 | transferencia | 2026-09-28 12:05:08.105 | PROCESADO | 48 | 2026-09-28 12:06:00.183 | 2026-09-28 12:06:05.202 |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

**Figura 45.** *FALLA-0001 en PROCESADO después de la recuperación*

```
Windows PowerShell - Laboratorio03_Pagos_Asyncronos 2026-09-28 12:06:10

PS C:\Laboratorio03_Pagos_Asyncronos> docker compose exec mysql mysql -t -u pagos_user -p pagos_db -e "SELECT p.id, p.referencia, p.valor, p.medio_pago, p.fecha_registro, p.estado, pr.
id AS proc_id, pr.fecha_toma, pr.fecha_procesamiento FROM pagos p LEFT JOIN procesamientos pr ON pr.pago_id = p.id WHERE p.referencia LIKE 'CAIDAK' ORDER BY p.id;"
+-----+-----+-----+-----+-----+-----+-----+-----+
| id | referencia | valor | medio_pago | fecha_registro | estado | proc_id | fecha_toma | fecha_procesamiento |
+-----+-----+-----+-----+-----+-----+-----+-----+
| 48 | CAIDA-0001 | 125000.00 | transferencia | 2026-09-28 12:05:28.040 | PROCESADO | 47 | 2026-09-28 12:05:50.407 | 2026-09-28 12:05:55.418 |
+-----+-----+-----+-----+-----+-----+-----+-----+

```

**Figura 46.** *CAIDA-0001 en PROCESADO después de la recuperación*

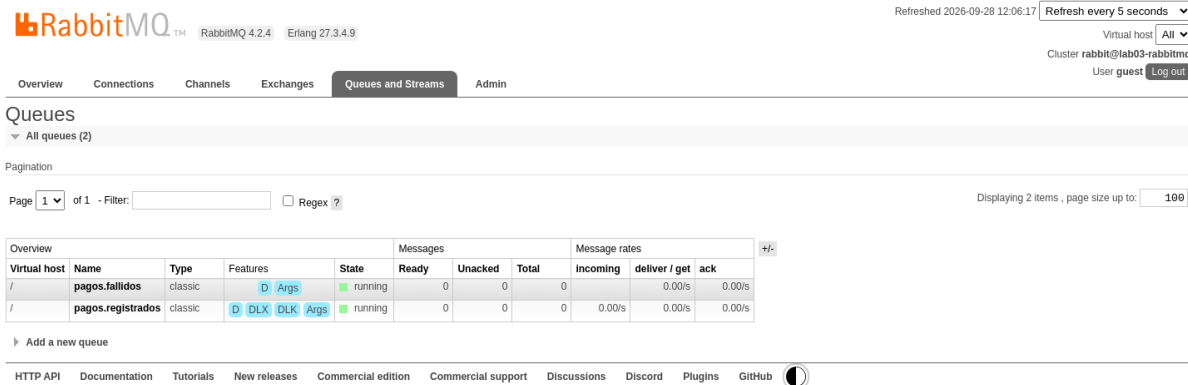


Figura 47. Consola de RabbitMQ al final: ambas colas vacías

## 7. Preguntas de cierre

### 1. ¿Cambió el tiempo de respuesta de la API al pasar la acción del consumidor de cinco a quince segundos? ¿Por qué?

No de forma significativa. Con la acción de 5 s la API respondió en promedio en 21,44 ms (máximo 63,39 ms), y con la acción de 15 s en 30,32 ms (máximo 77,84 ms). La media sí fue 8,88 ms mayor en la segunda corrida, pero esa diferencia no guarda relación con la acción: es más de mil veces menor que los 10 000 ms que se añadieron y es del mismo orden que la variación entre corridas (con el consumidor totalmente apagado la media fue 28,54 ms). Si la API esperara al consumidor, cada respuesta habría subido al menos 10 s. En cambio, el tiempo hasta que el último pago quedó PROCESADO pasó de 101,081 s a 312,222 s. El motivo es que la API sólo hace trabajo propio (validar, un INSERT con COMMIT y publicar un mensaje confirmado por el broker) y responde; nunca espera al consumidor. La duración de la acción afecta únicamente al lado del consumidor: cuánto tiempo pasan los mensajes en la cola.

### 2. Con el consumidor detenido, la API respondió que el pago fue registrado. ¿Qué le está prometiendo esa respuesta al cliente y qué no le está prometiendo?

Le promete que el pago fue aceptado como válido, que quedó guardado de forma durable en la base de datos con estado REGISTRADO y un identificador para consultarlo, y que la solicitud de procesamiento quedó encolada de forma persistente (confirmada por RabbitMQ), así que se procesará cuando haya un consumidor disponible. En el caso 1 los cinco pagos se registraron (media 28,54 ms) aunque no había consumidor, y se procesaron todos al levantarlo. No le promete que el pago ya fue procesado, ni cuándo lo será, ni que el procesamiento vaya a salir bien: el estado PROCESADO y el comprobante llegan después, y el cliente debe consultarlos con GET /pagos/{id}.

### 3. Si este sistema tuviera un plazo declarado, ¿sobre qué parte del flujo se podría declarar y sobre cuál no? Justifique con sus mediciones.

Se puede declarar sobre la parte síncrona: la respuesta de la API (validar, guardar y encolar). En todas las mediciones fue estable y acotada: media de 21,44 ms y máximo de 63,39 ms con acción de 5 s, media de 30,32 ms y máximo de 77,84 ms con acción de 15 s, y media de 28,54 ms incluso con el consumidor apagado. Con esos datos un plazo como “la API responde en menos de 200 ms” se cumple con amplio margen.

No se puede declarar, con este diseño, sobre el tiempo hasta que el pago queda PROCESADO, porque depende de factores que la API no controla: cuántos mensajes hay

antes en la cola, cuánto dura la acción y si el consumidor está en marcha. En las mediciones el primer pago tardó 5,036 s, pero el último 101,081 s con 5 s y 312,222 s con 15 s. Con el consumidor detenido la espera no tiene límite: los pagos del caso 1 esperaron hasta que se levantó el consumidor. Un plazo sobre el procesamiento sólo podría declararse acotando la carga y la duración de la acción, y garantizando consumidores disponibles.

## 8. Entregables

| Entregable                                  | Ubicación                                                                                  |
|---------------------------------------------|--------------------------------------------------------------------------------------------|
| Código fuente de la API                     | api/app.py                                                                                 |
| Código fuente del consumidor                | consumer/consumer.py, consumer/reprocesar_fallidos.py                                      |
| Script de creación de las tablas            | db/init.sql                                                                                |
| Dockerfile y Docker Compose                 | api/Dockerfile, consumer/Dockerfile, herramientas/Dockerfile, docker-compose.yml           |
| Instructivo de instalación                  | README.md                                                                                  |
| Documentación de la API (Swagger / OpenAPI) | <a href="http://localhost:8090/docs">http://localhost:8090/docs</a> y swagger/openapi.json |
| Tabla de medición y datos crudos            | Sección 5 y resultados/*.json                                                              |
| Evidencias de los tres casos                | Sección 6 y evidencias/capturas/                                                           |
| Consulta del mismo pago antes y después     | Sección 4.2                                                                                |
| Preguntas de cierre                         | Sección 7                                                                                  |